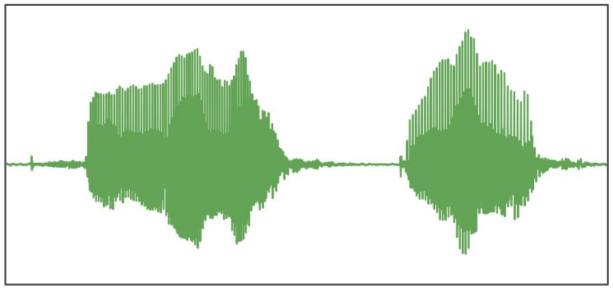
Automatic Speech Recognition
A.k.a. Speech-To-Text

Grinberg Petr
HSE, 2025

Uses materials from https://github.com/markovka17/dla

Automatic Speech Recognition

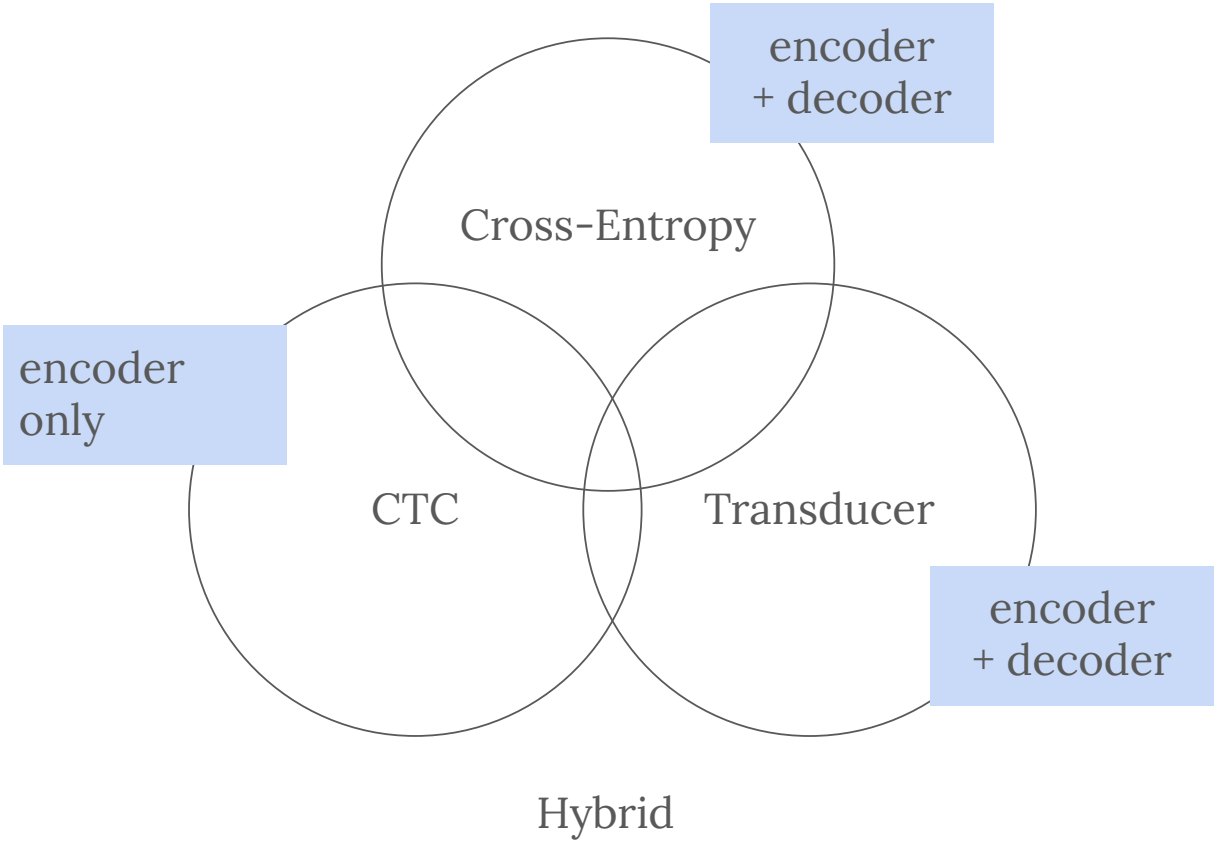


→ **hello world**

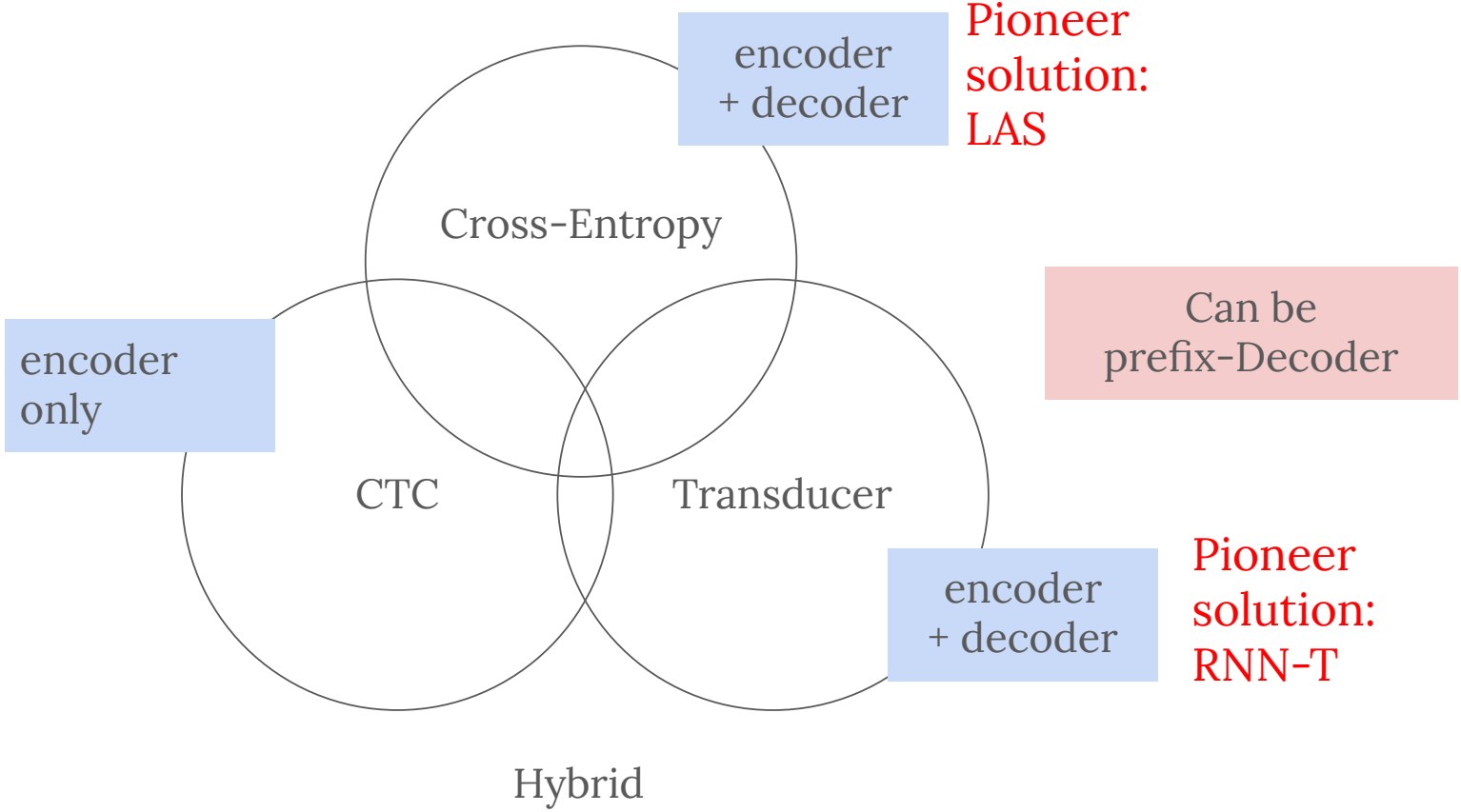
Applications:

- Speech Transcription (For meetings, e.g.)
- Voice Dictation
- Voice Assistant control
- Agents (LLMs) control
- Speech Understanding*
- Speech Translation*

Types of ASR models



Types of ASR models



Metrics

Target: the quick brown fox jumps over a lazy dog

Prediction: the quick brow an fox jumps over lazy dog

How to measure distance between sentences?

Metrics

Target: the quick brown fox jumps over a lazy dog

Prediction: the quick brow an fox jumps over lazy dog

How to measure distance between sentences?

Edit path from reference to prediction:

1. S - substitutions
2. D - deletions
3. I - insertions

Metrics

Target: the quick brown fox jumps over a lazy dog

Prediction: the quick brow an fox jumps over lazy dog

How to measure distance between sentences?

Edit path from reference to prediction:

1. S - substitutions
2. D - deletions
3. I - insertions
4. N - total words in target

Word Error Rate (WER) - normalized Levenshtein Distance.

$$WER = \frac{S + D + I}{N}$$

Metrics

Target: the quick brown fox jumps over a lazy dog

Prediction: the quick brow an fox jumps over lazy dog

How to measure distance between sentences?

Edit path from reference to prediction:

1. S - substitutions
2. D - deletions
3. I - insertions
4. N - total words in target

Word Error Rate (WER) - normalized Levenshtein Distance.

$$WER = \frac{S + D + I}{N}$$

CER - Character Error Rate

.. the same thing, but at the character level.

Metrics

Target: the quick brown fox jumps over a lazy dog

Prediction: the quick brow an fox jumps over lazy dog

How to measure distance between sentences?

Edit path from reference to prediction:

1. S - substitutions
2. D - deletions
3. I - insertions
4. N - total words in target

Word Error Rate (WER) - normalized Levenshtein Distance.

$$WER = \frac{S + D + I}{N}$$

CER - Character Error Rate

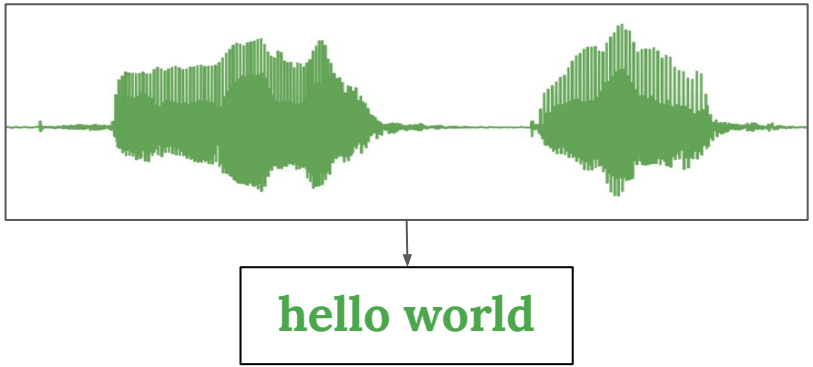
.. the same thing, but at the character level.

Questions:

1. What is more difficult to minimize (WER or CER)?
2. WER > 1 ?
3. N = 0 ?

What do we want?

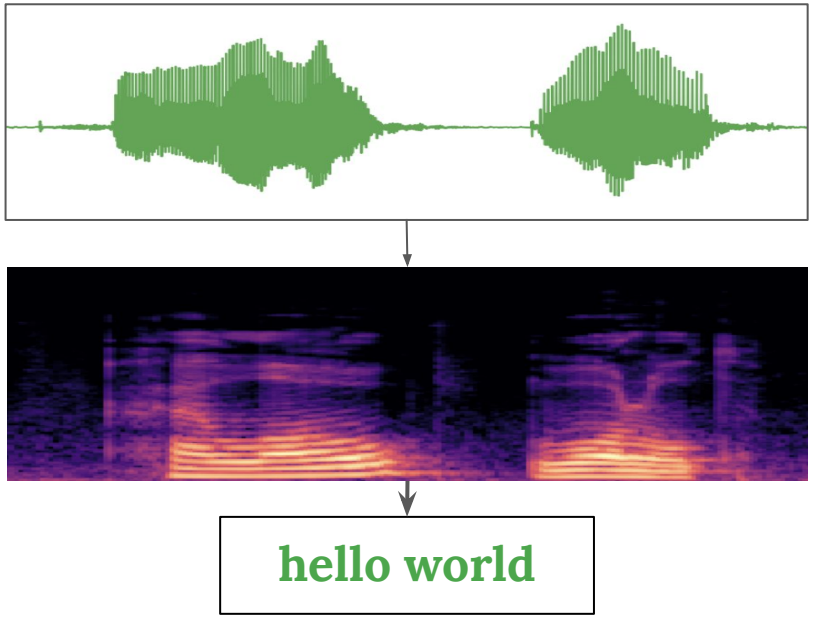
- build an asr model



What do we want?

- build an asr model

- we have:
  - variable input lengths $x_1\ x_2\ ..\ x_t$
  - variable output length$y_1\ y_2\ ..\ y_u\ ,\ t\ \geq\ u$

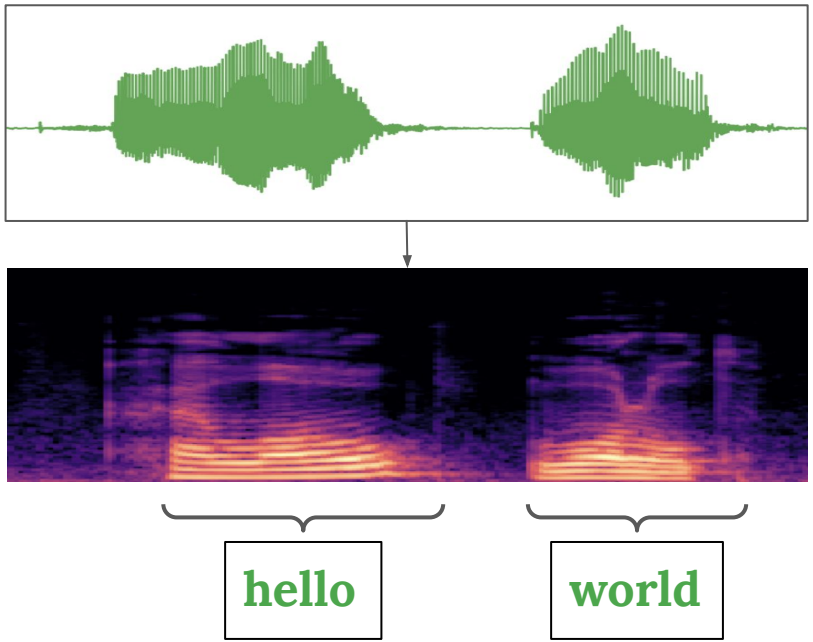


What do we want?

- build an asr model

- we have:
  - variable input lengths $x_1\ x_2\ ..\ x_t$
  - variable output length$y_1\ y_2\ ..\ y_u\ ,\ t\ \geq\ u$

- **problem**:
  we don't have an alignment between x and y

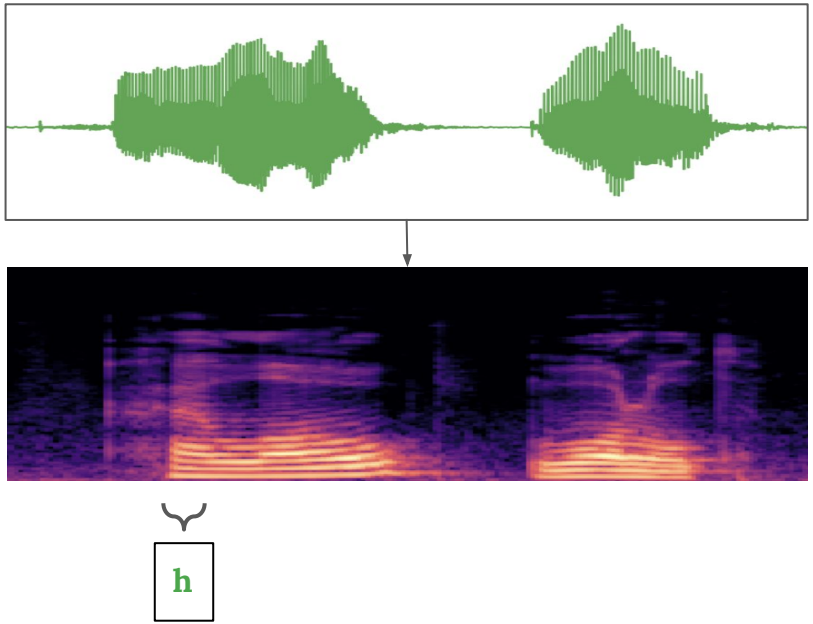


What do we want?

- build an asr model

- we have:
  - variable input lengths $x_1\ x_2\ ..\ x_t$
  - variable output length$y_1\ y_2\ ..\ y_u\ ,\ t \geq u$

- **problem**:
  we don't have an alignment between x and y

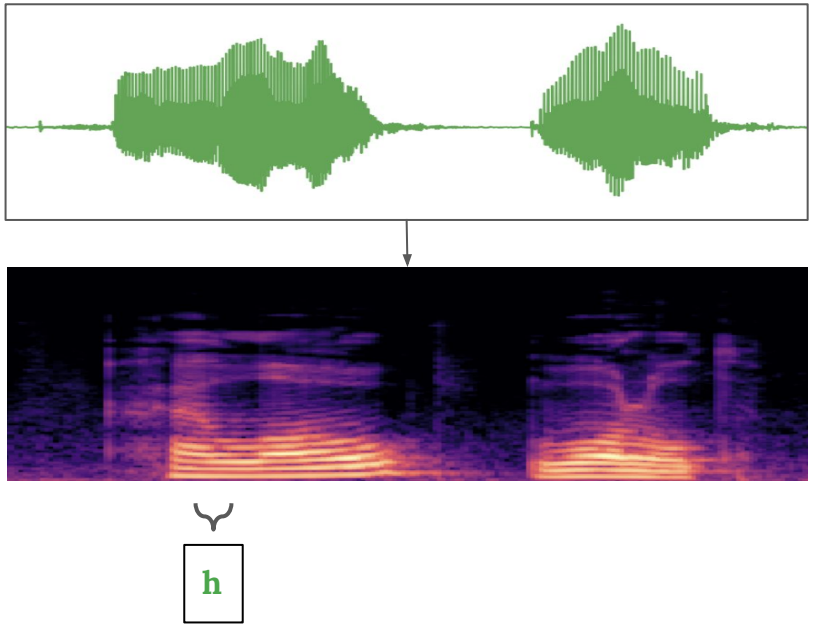


What do we want?

- build an asr model

- we have:
  - variable input lengths $x_1\ x_2\ ..\ x_t$
  - variable output length$y_1\ y_2\ ..\ y_u\ ,\ t\ \geq\ u$

- **problem**:
  we don't have an alignment between x and y

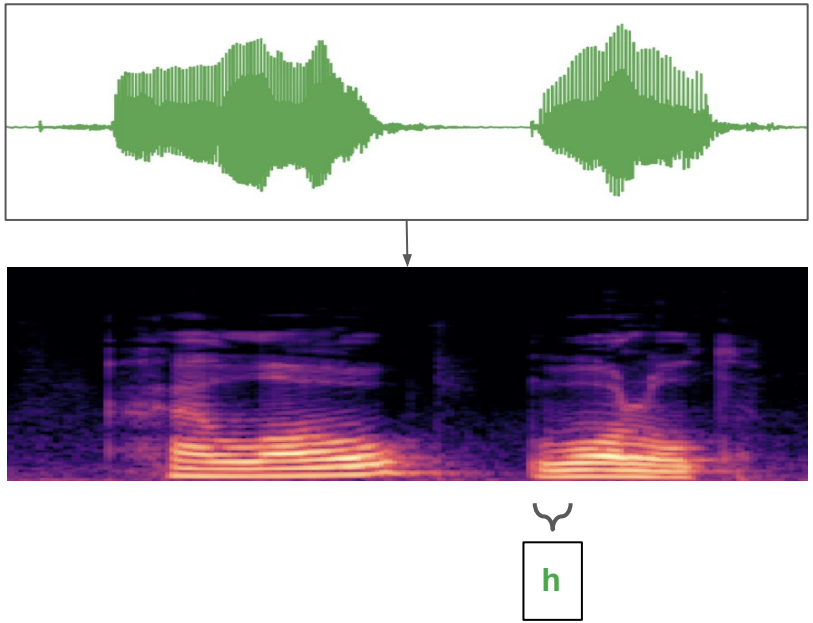


What do we want?

- build an asr model

- we have:
  - variable input lengths $x_1\ x_2\ ..\ x_t$
  - variable output length$y_1\ y_2\ ..\ y_u$, $t \geq u$

- **problem**:
  we don't have an alignment between x and y



Naive Approach

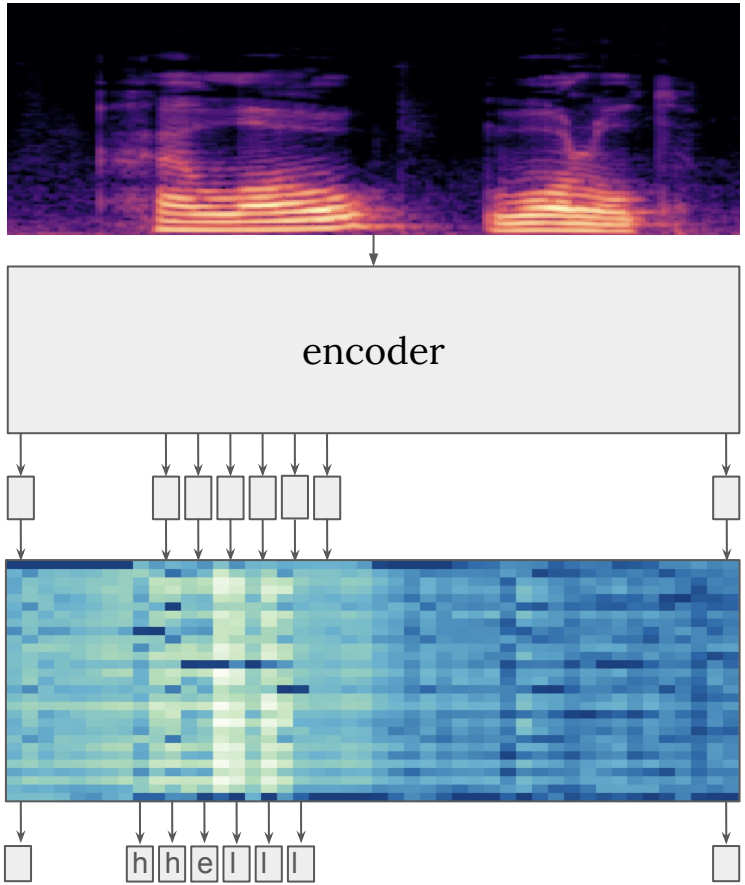
- build a model that predicts probability distribution for each x_j over vocabulary V

$$O = P(V||X) = Model(X)$$
$$X \in \mathbb{R}^{seq\_len \times n\_mel}, O \in \mathbb{R}^{seq\_len \times vocab\_size}$$

- then, we can easily predict output as argmax over the time axis

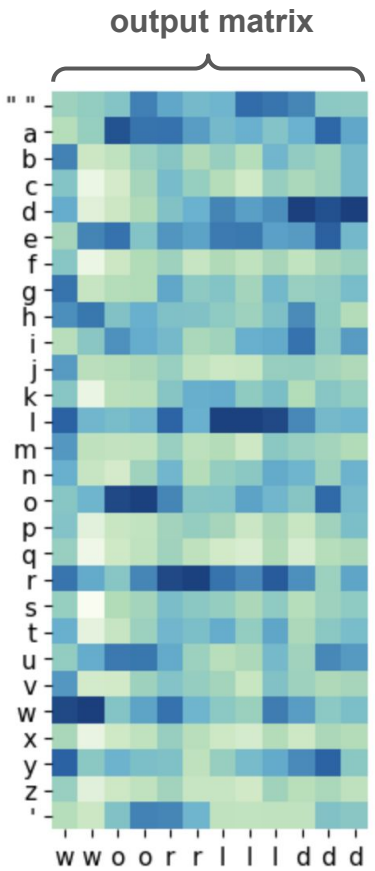
$$Y^* = \arg\max O, Y \in \mathbb{R}^{seq\_len}$$

$$Y^* = \{h, h, e, l, l, l, ..\}$$



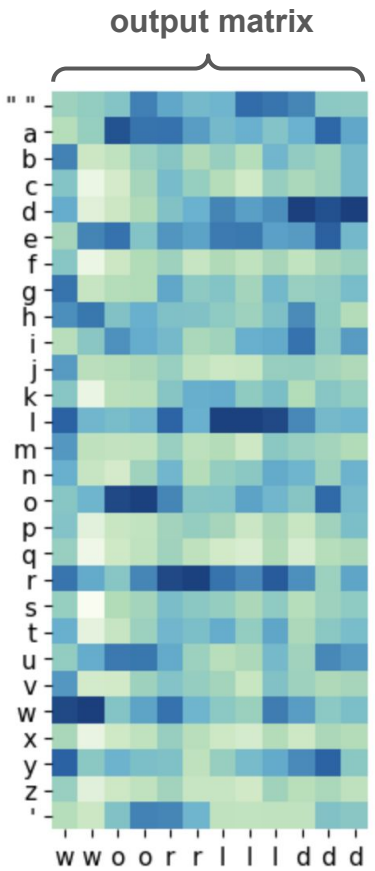
What do we want?

- thus, we have to decode $Y^* \in \mathbb{R}^{seq\_len}$, but how?



What do we want?

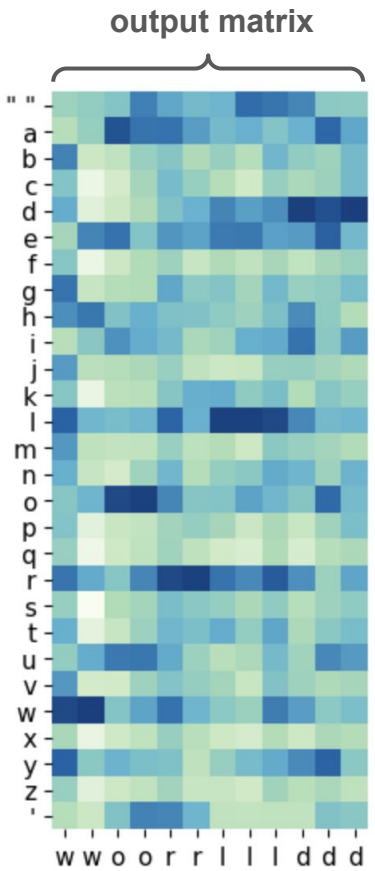
- thus, we have to decode $Y^* \in \mathbb{R}^{seq\_len}$, but how?
  - merge consecutive letters



What do we want?

- thus, we have to decode $Y^* \in \mathbb{R}^{seq\_len}$, but how?
  - merge consecutive letters

- issues?
  - multiple consecutive letters in a target word (e.g. **hello)**
  - silence between words and letters (e.g. breathing, lip-smacking)



What do we want?

- add special symbol to vocabulary - **blank symbol** - ε
- train model in such a way, to predict blank symbol in issue cases
- how to deal with blank symbol while decoding?
  - merge all repeated symbols into one

    h ε e l l ε ε l ε o → h ε e l ε l ε o
  - delete blanks

    h ε e l ε l ε o → h e l l o

How should we define a loss?

- we have valid paths that decode into the target

h	ε	e	l	ε	l	ε	o	o	o
h	ε	e	l	l	l	ε	l	o	o
ε	h	h	h	e	l	ε	l	o	o

- and non-valid paths

h	ε	e	**l**	**ε**	**l**	**ε**	**l**	o	o
h	ε	e	l	ε	l	ε	o		
ε	h	h	h	e	l	ε	l	**ε**	**ε**

How should we define a loss?

- we have valid paths that decode into the target

h	ε	e	l	ε	l	ε	o	o	o
h	ε	e	l	l	l	ε	l	o	o
ε	h	h	h	e	l	ε	l	o	o

- we don't care what path the model selects, thus, we maximize probabilities for all of them

How should we define a loss?

- we have valid paths that decode into the target

h	ε	e	l	ε	l	ε	o	o	o
h	ε	e	l	l	l	ε	l	o	o
ε	h	h	h	e	l	ε	l	o	o

- we don't care what path the model selects, thus, we maximize probabilities for all of them

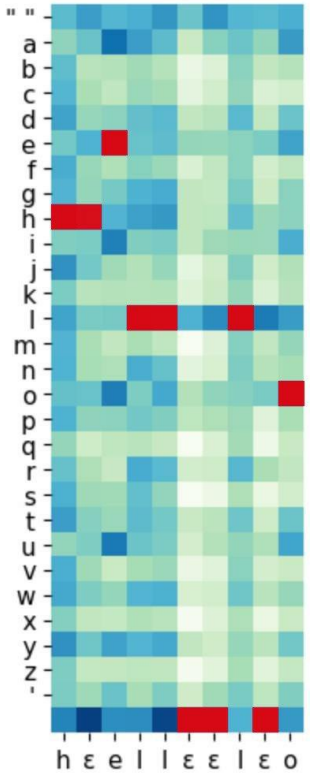
- for each valid path we can compute its probability from the output matrix

How should we define a loss?

- we don't care what path the model selects, thus we maximize probabilities for all of them

- for each valid path we can compute it's probability from the output matrix

P(h h e l l ε ε l ε o) = 0.00123



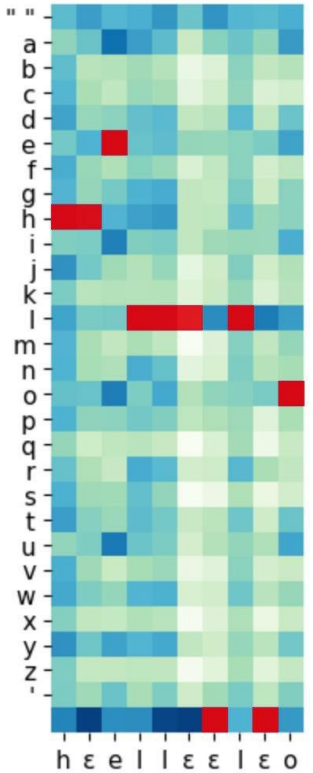
How should we define a loss?

- we don't care what path the model selects, thus we maximize probabilities for all of them

- for each valid path we can compute it's probability from the output matrix

P(h h e l l ε ε l ε o) = 0.00123

P(h h e l l l ε l ε o) = 0.00112



How should we define a loss?

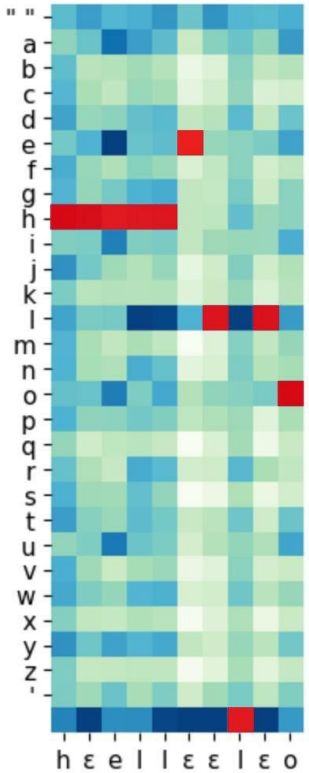
- we don't care what path the model selects, thus we maximize probabilities for all of them

- for each valid path we can compute it's probability from the output matrix

P(h h e l l ε ε l ε o) = 0.00123

P(h h e l l l ε l ε o) = 0.00112

P(h h h h h e l ε l o) = 0.00001



How should we define a loss?

- we don’t care what path the model selects, thus we maximize probabilities for all of them

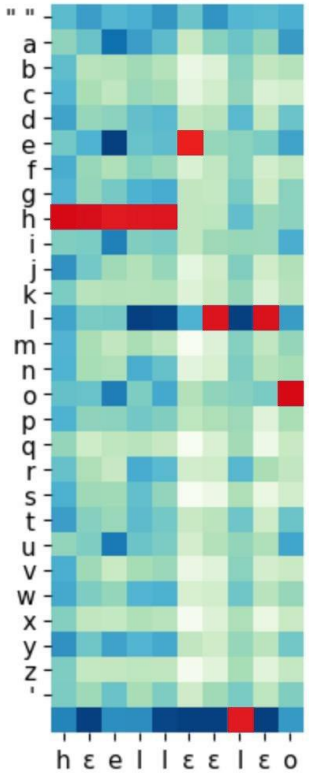
- for each valid path we can compute it’s probability from the output matrix

P(h h e l l ε ε l ε o) = 0.00123

P(h h e l l l ε l ε o) = 0.00112

P(h h h h h e l ε l o) = 0.00001

- and for many many others…



CTC Loss

- we don't care what path the model selects, thus we maximize probabilities for all of them

- for each valid path we can compute it's probability from the output matrix

P(h h e l l ε ε l ε o) = 0.00123

P(h h e l l l ε l ε o) = 0.00112

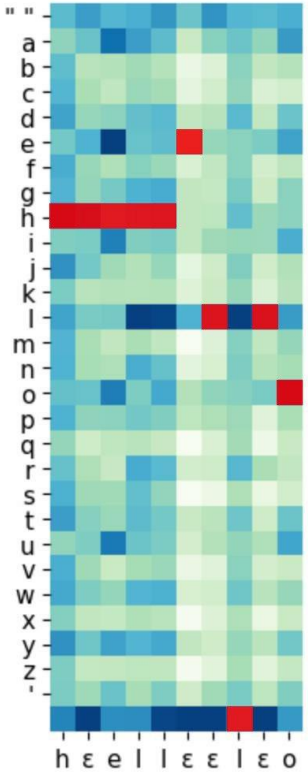
P(h h h h h e l ε l o) = 0.00001

$$p(Y \mid X) = \sum_{A \in \mathcal{A}_{X,Y}} \prod_{t=1}^{T} p_t(a_t \mid X)$$

The CTC conditional **probability**

marginalizes over the set of valid alignments

computing the **probability** for a single alignment step-by-step.



https://distill.pub/2017/ctc/

Efficient CTC Loss Compute

$$\text{target} = \text{ab}$$

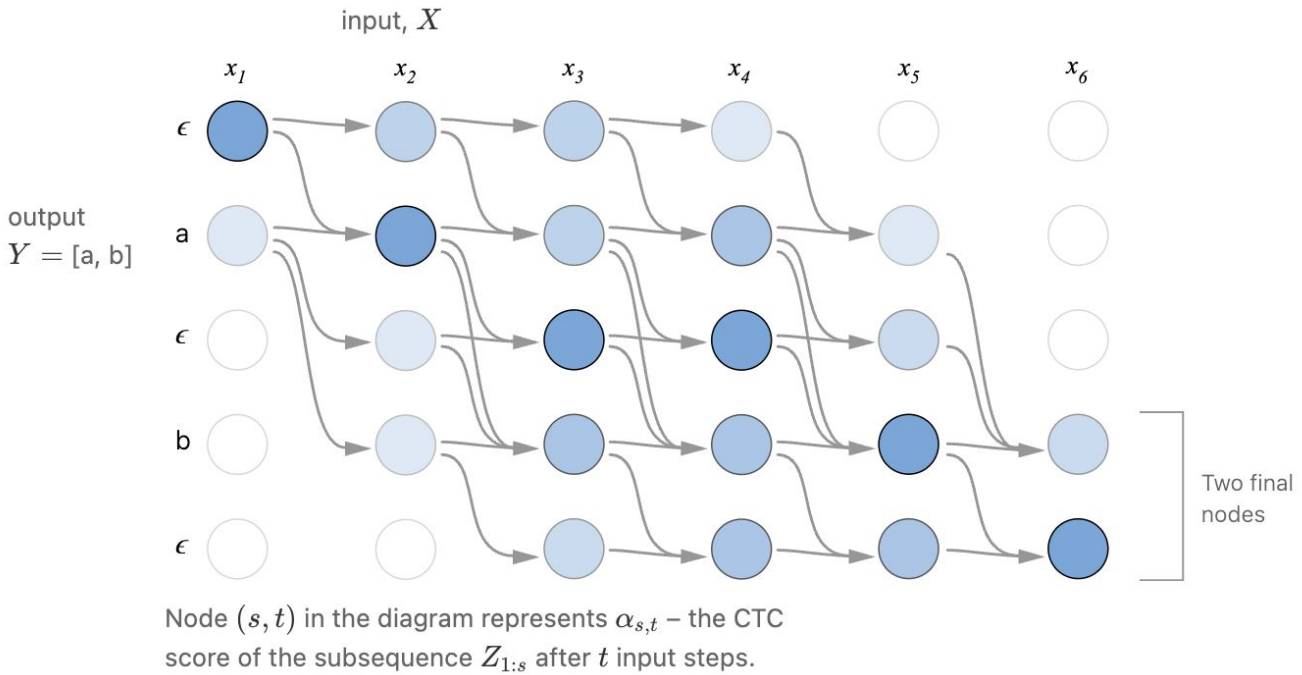
$$p(\text{out=ab})_T = p(\text{out=ab, last\_char} = \text{b})_T + p(\text{out=ab, last\_char} = \varepsilon)_T$$

$$\begin{aligned} p\left(\text{out=ab, last\_char} = \text{b}\right)_T = \\ p(\text{prefix=a, last\_char} = \text{b})_{T-1} \cdot p(\text{char=b})_T \\ + p(\text{prefix=a, last\_char} = \varepsilon)_{T-1} \cdot p(\text{char=b})_T \\ + p(\text{prefix=a, last\_char} = \text{a})_{T-1} \cdot p(\text{char=b})_T \end{aligned}$$

$$\begin{aligned} p\left(\text{out=ab, last\_char} = \varepsilon\right)_T = \\ p(\text{prefix=a, last\_char} = \text{b})_{T-1} \cdot p(\text{char=}\varepsilon)_T \\ + p(\text{prefix=ab, last\_char} = \varepsilon)_{T-1} \cdot p(\text{char=}\varepsilon)_T \end{aligned}$$

Efficient CTC Loss Compute

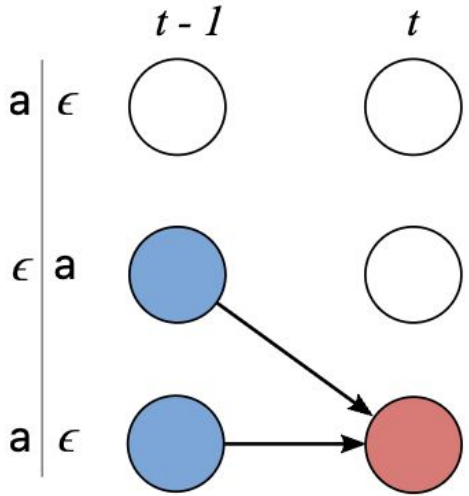
$$Z = [\epsilon, y_1, \epsilon, y_2, \ldots, \epsilon, y_U, \epsilon]$$



Node (s, t) in the diagram represents $\alpha_{s,t}$ – the CTC score of the subsequence $Z_{1:s}$ after t input steps.

https://distill.pub/2017/ctc/

Efficient CTC Loss Compute - Case 1



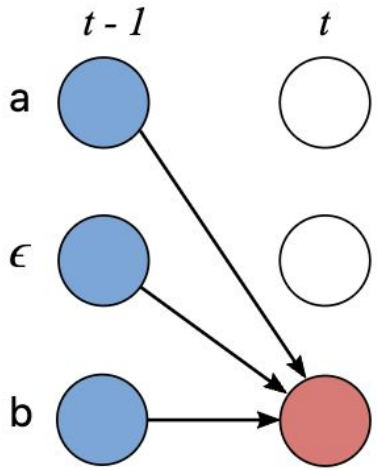
$$\alpha_{s,t} = (\alpha_{s-1,t-1} + \alpha_{s,t-1}) \cdot p_t(z_s \mid X)$$

The CTC probability of the two valid subsequences after $t-1$ input steps.

The probability of the current character at input step t.

https://distill.pub/2017/ctc/

Efficient CTC Loss Compute - Case 2



$$\alpha_{s,t} = (\alpha_{s-2,t-1} + \alpha_{s-1,t-1} + \alpha_{s,t-1}) \cdot p_t(z_s \mid X)$$

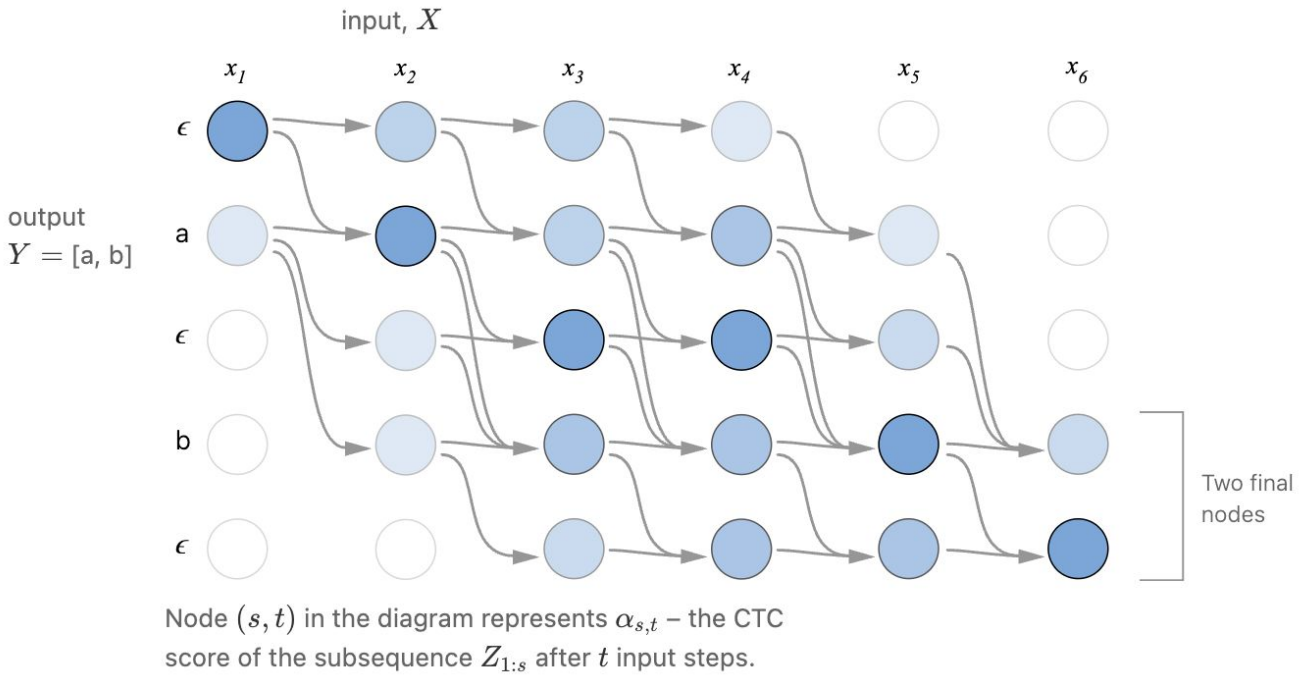
The CTC probability of the three valid subsequences after $t-1$ input steps.

The probability of the current character at input step t.

https://distill.pub/2017/ctc/

Efficient CTC Loss Compute

$$Z = [\epsilon, y_1, \epsilon, y_2, \ldots, \epsilon, y_U, \epsilon]$$



Node (s, t) in the diagram represents $\alpha_{s,t}$ – the CTC score of the subsequence $Z_{1:s}$ after t input steps.

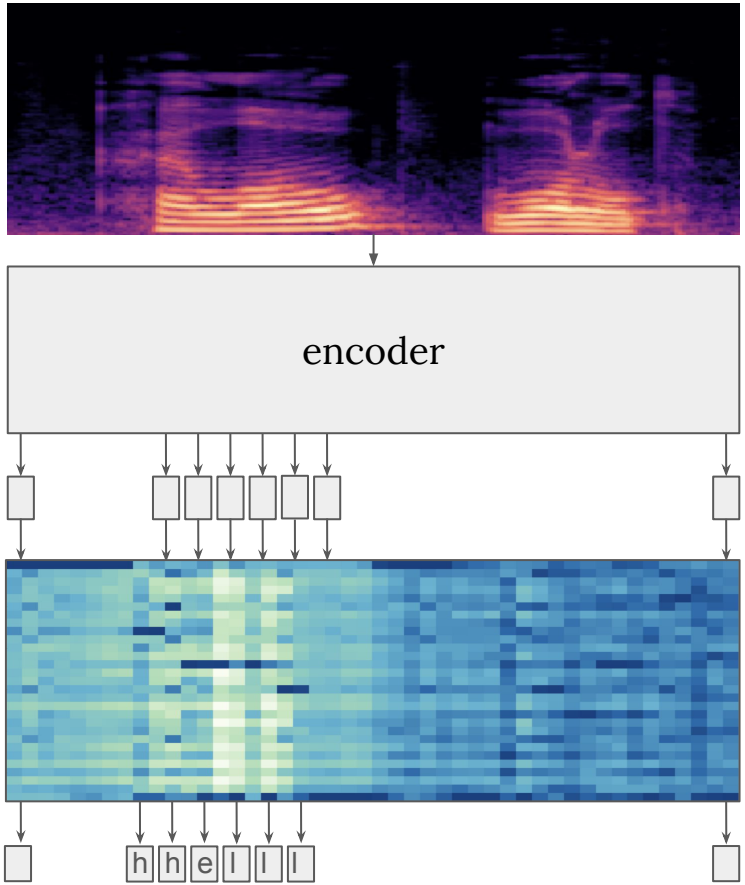
https://distill.pub/2017/ctc/

CTC Properties

- (+) **can be streamable**: we don't need full audio, to start predicting

- (-) **no language context -** predicted labels are independent, meaning that each letter does not know about others

- (+) **produces alignment** between audio and target



CTC Inference

- argmax is fast, but:

  Problems:

  ```
  P([b,b,b]) = 0.5 <- argmax
  P([a,a,ϵ]) = 0.4
  P([a,a,a]) = 0.2
  ```

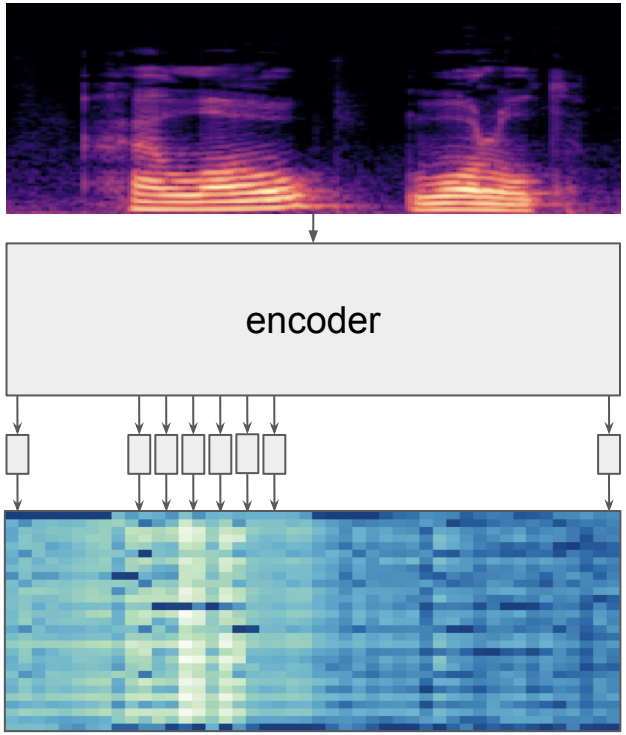
  but

  ```
  P("a") = 0.6
  P("b") = 0.5
  ```

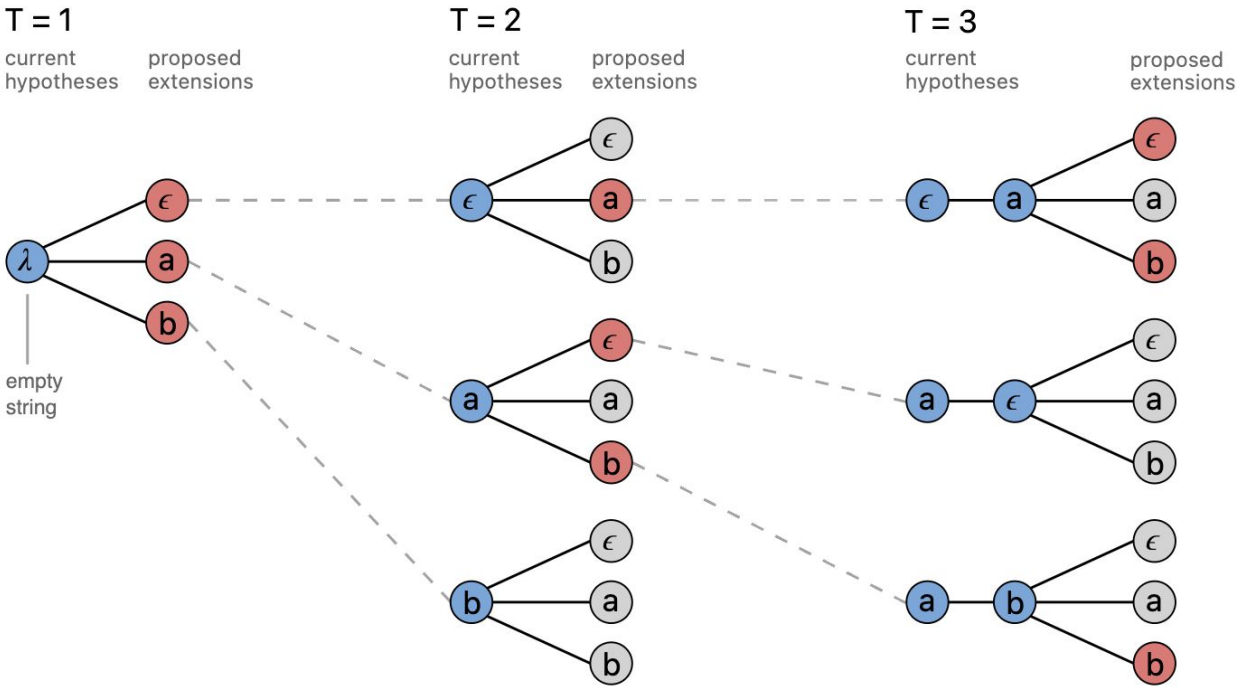
- we can't compute all paths

CTC Beam Search

- Want to decode an output matrix into a list of text predictions with probabilities
- Has parameter: beam size >= 1
- Tradeoff between:
  - taking argmax prediction - easy and fast to compute, but not perfectly accurate
  - computing sum of probabilities of all possible paths - perfect quality, but infeasible



Beam Search

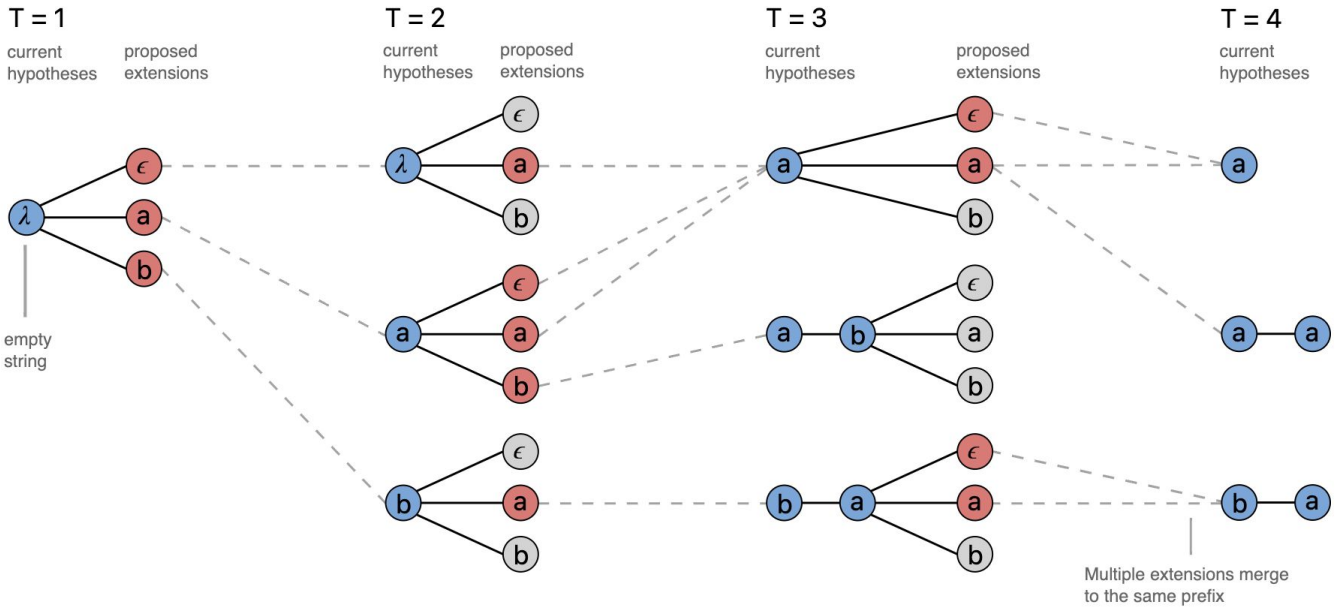


A standard beam search algorithm with an alphabet of $\{\epsilon, a, b\}$ and a beam size of three.

Beam search step:
- expand beam
- truncate beam

https://distill.pub/2017/ctc/

CTC Beam Search



The CTC beam search algorithm with an output alphabet $\{\epsilon, a, b\}$ and a beam size of three.

Beam search step:
- expand beam
- merge paths
- truncate beam

https://distill.pub/2017/ctc/

CTC* Summary

*** Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks, A. Graves et. al, 2006**

CTC Properties

- conditional independence
- streamable
- produces alignment
- fast inference

CTC Decoding

- allows to reduce fixed len predictions to variable length predictions
- provides mapping from path to string

How to train:

- compute encoder predictions - output matrix
- efficiently compute sum of probs of all paths corresponding to target
- maximize this sum

How to evaluate:

- compute encoder predictions - output matrix
- argmax / beam search
- CTC decode

Beam Search:

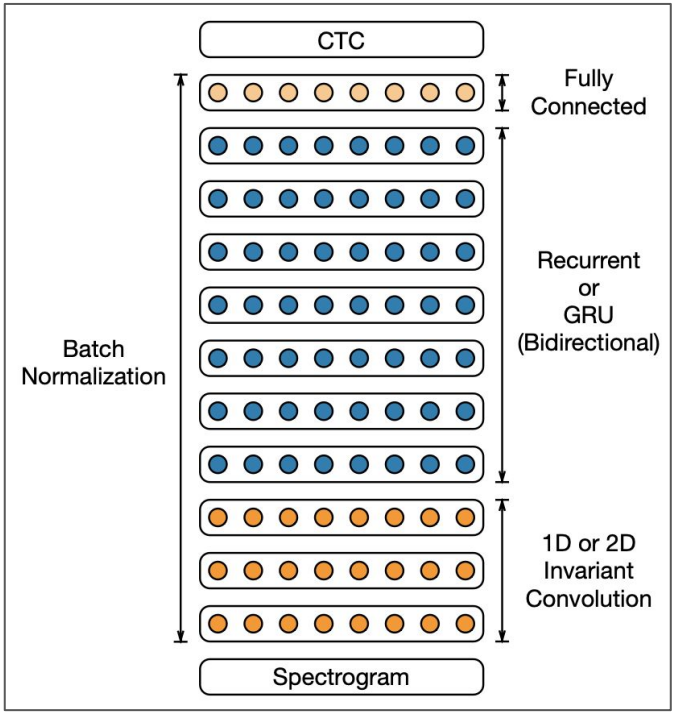
- expand beam
- merge paths
- truncate beam
- repeat

Encoders: Deep Speech 2

- **Deep Speech 2: End-to-End Speech Recognition in English and Mandarin**, Dario Amodei et al., 2015

- 11,9k hours of training data
- 35M parameters
- no torch in 2015 - many challenges
- super human quality on clean sets
- 3-5 days on 16 GPUs

Read Speech			
Test set	DS1	DS2	Human
WSJ eval'92	4.94	3.60	5.03
WSJ eval'93	6.94	4.98	8.08
LibriSpeech test-clean	7.89	5.33	5.83
LibriSpeech test-other	21.74	13.25	12.69

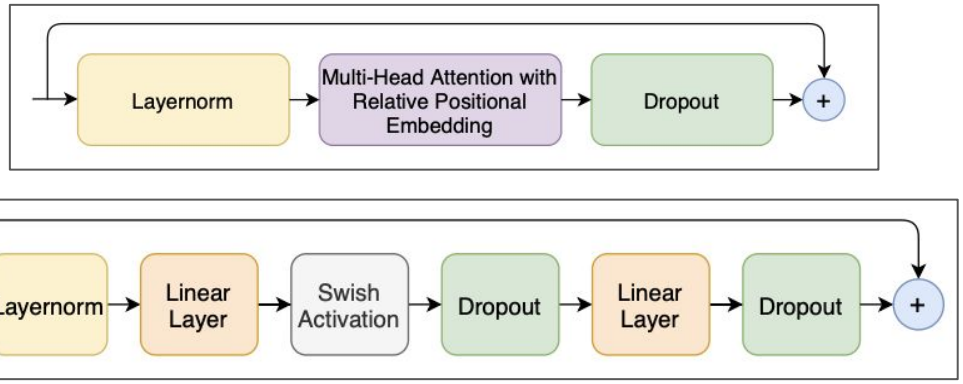


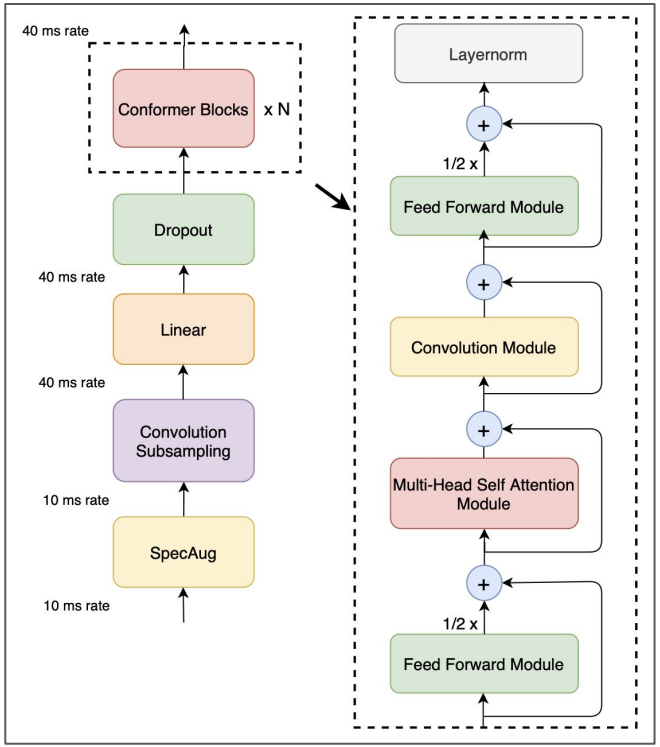
https://arxiv.org/abs/1512.02595

Encoders: Conformer

- **Conformer: Convolution-augmented Transformer for Speech Recognition,** Anmol Gulati, Google, 2020

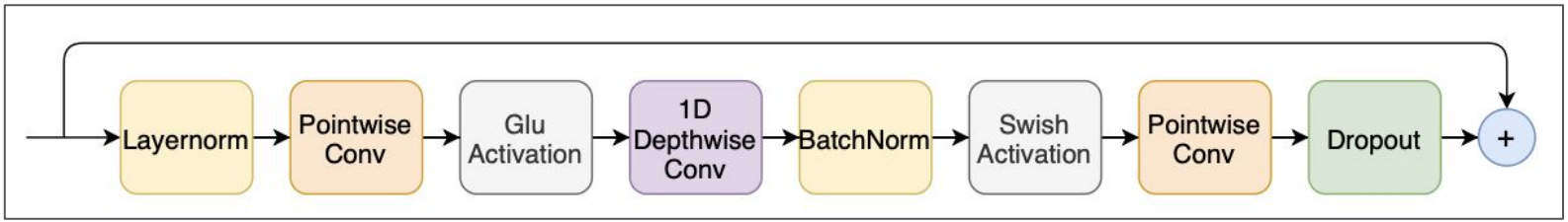
- transformers
- 1D depthwise separable convolution
- convolutions for local, attention for global dependencies





https://arxiv.org/abs/2005.08100

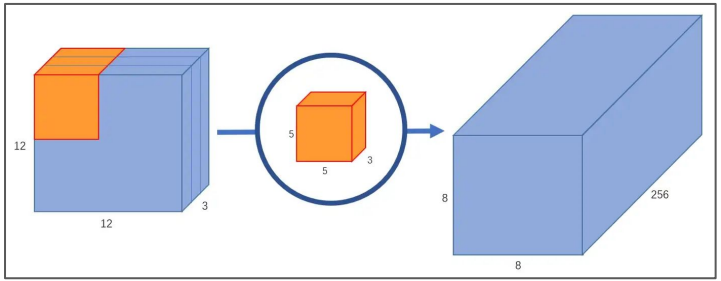
Encoders: Conformer Convolution



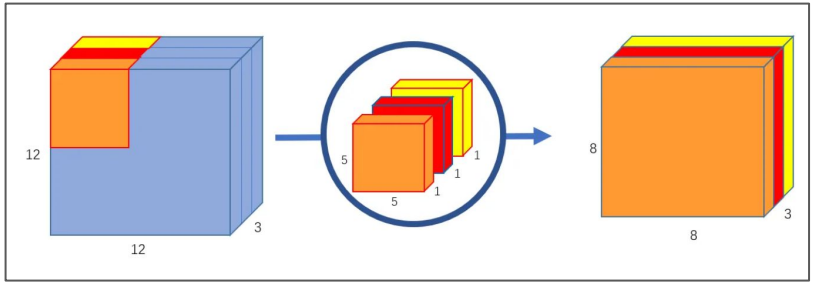
https://arxiv.org/abs/2005.08100

Encoders: Conformer Convolution

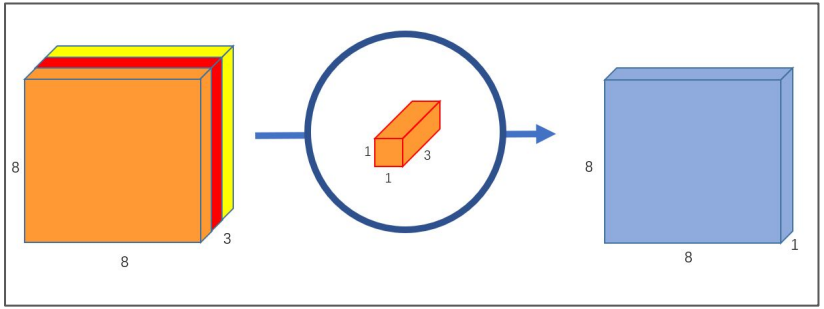
Standard Convolution



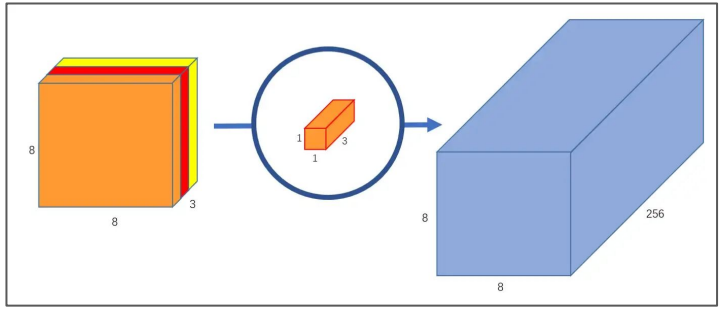
Depthwise Convolution



Pointwise Convolution



Depthwise Separable Convolution



https://github.com/youssefHosni/

Subword Tokenization

- motivation:
  - a lot of characters have different pronunciation in different contexts (e.g. **ch**eese and **ch**aracter)
  - generated tokens are conditionally independent of each other (in case of CTC) (e.g. reading)
  - reducing the target sentence length by subword tokenization, we are trying to sidestep the CTC loss sequence length limitation
  - less compute leads to faster training and inference
  - better generalization leads to better quality

- subword tokenization:
  - Byte Pair Encoding (BPE)
  - Word Piece
  - Unigram

BPE

Algorithm 1 Byte-pair encoding [30]

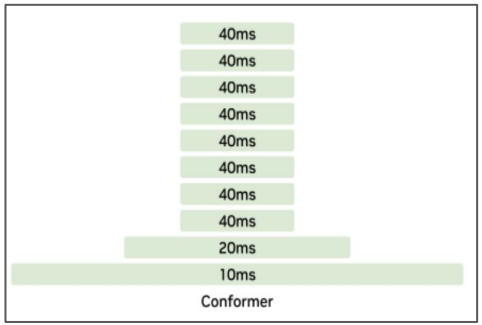
Input: set of strings D, target vocab size K
procedure BPE(D, K)
    $V \leftarrow$ all unique characters in D
    while $|V| < K$ **do** ▷ *Merge tokens*
        $t_L, t_r \leftarrow$ Most frequent bigram in D
        $t_{NEW} \leftarrow t_L + t_R$ ▷ *Make new token*
        $V \leftarrow V + [t_{NEW}]$
        Replace each occurrence of t_L, t_r in D with t_{NEW}
    end while
    return V
end procedure

```
    aaabdaaabac

1.  add Z <- aa

    Zabd Zabac
2.  add Y <- ab

    ZYd ZYac
3.  add X <- ZY

    XdXac
```

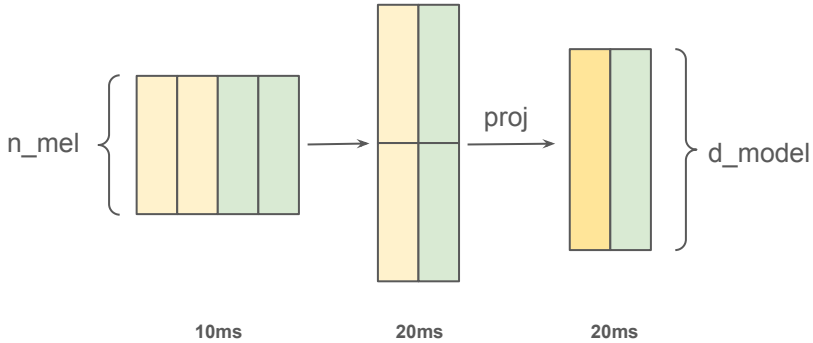

https://skemman.is/bitstream/ADsar-DavidErikMollberg.pdf

Subsampling

- motivation:
  - speed up training and inference
  - subword tokenization can work better, because we will encode more data per frame



- options:
  - 1d conv with stride
  - 2d conv and projection
  - stacking consecutive frames and projection
  - and many others..



https://arxiv.org/abs/2305.05084

Language models (LM): motivation

motivation:

- spelling of a word heavily depends on its context
- LMs can add context dependency in CTC beam search
- external language models typically saw more data

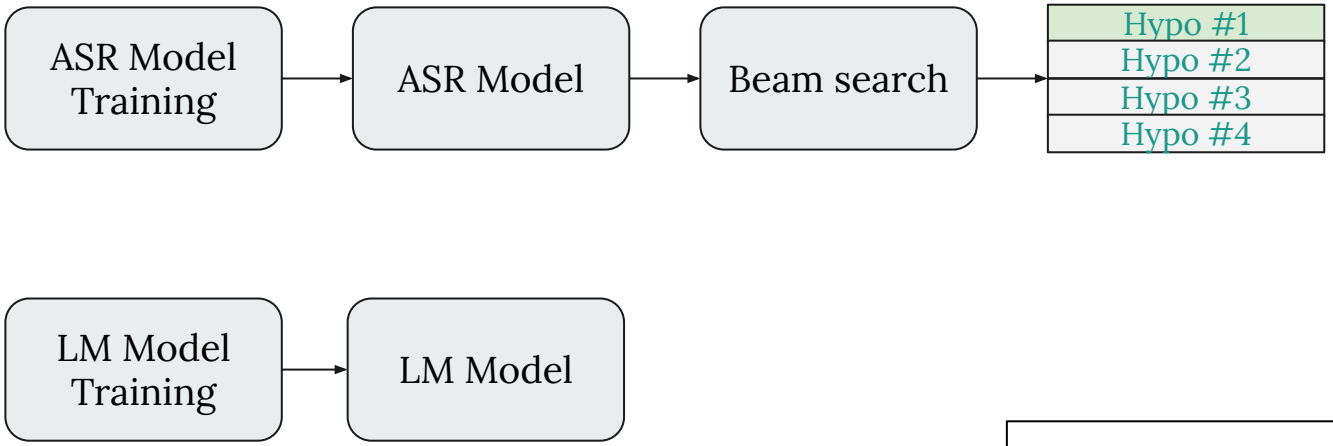
examples:

- simple: n-gramms, Kneser–Ney smoothing and others
- complex: neural networks e.g: Bert, GPT, LLaMA

hypo 1: let's go ***two*** a movie (am score: 0.21)
hypo 2: let's go ***to*** a movie (am score: 0.19)
hypo 3: let's go ***too*** a movie (am score: 0.13)

P(let's go ***two*** a movie) = 0.01 (lm score)
P(let's go ***to*** a movie) = 0.6 (lm score)

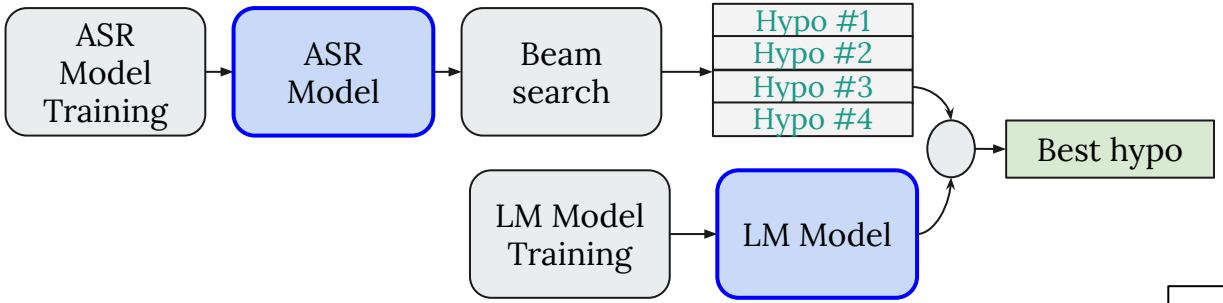
LM



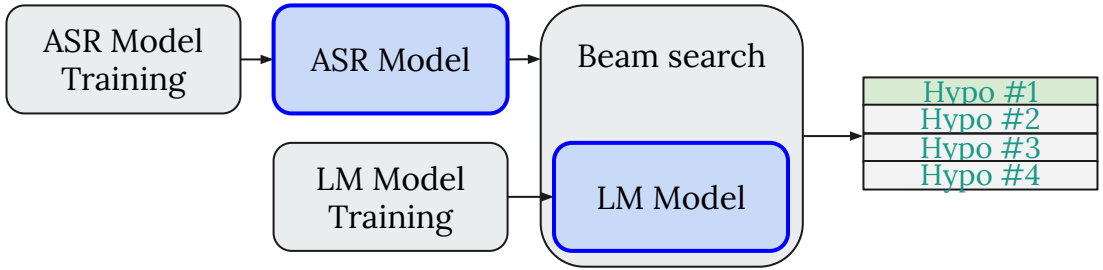
How to integrate LM?

LM

Second pass rescoring:



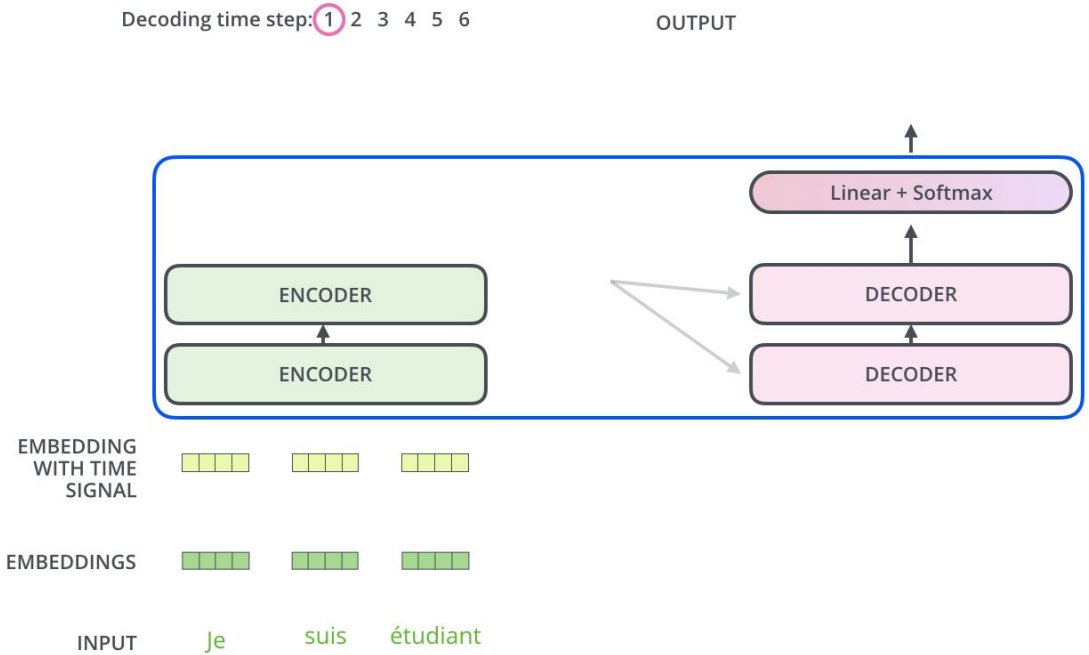
Shallow fusion:



it's better to use:
- large model for rescoring
- light model for shallow fusion

Does the modality matter?

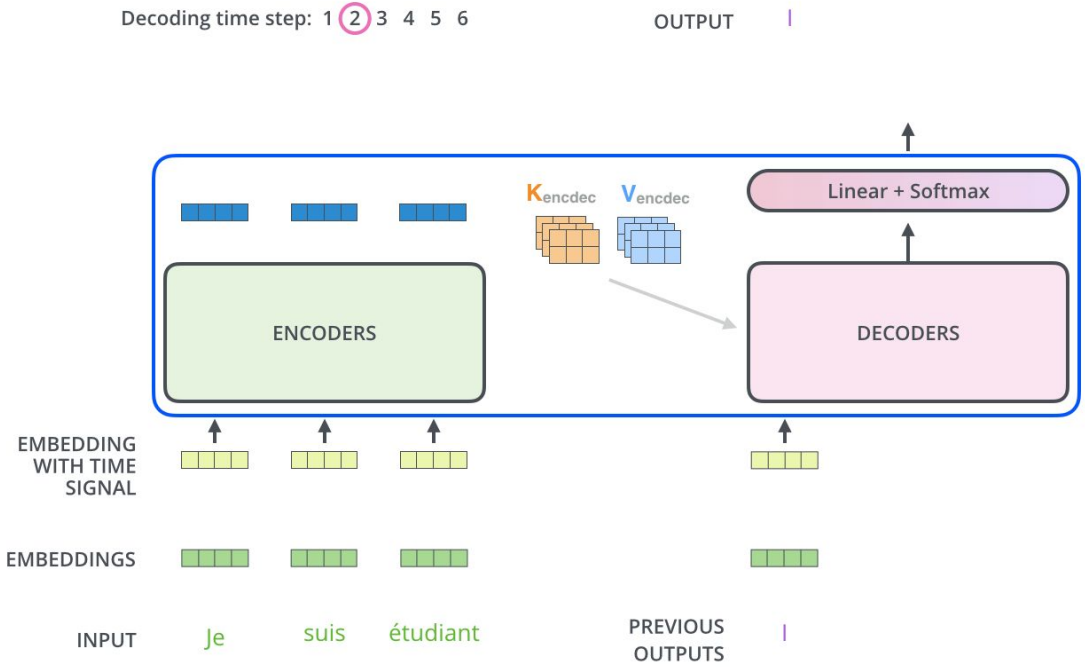
Let's recall how text generation worked:



Source: illustrations from https://jalammar.github.io/

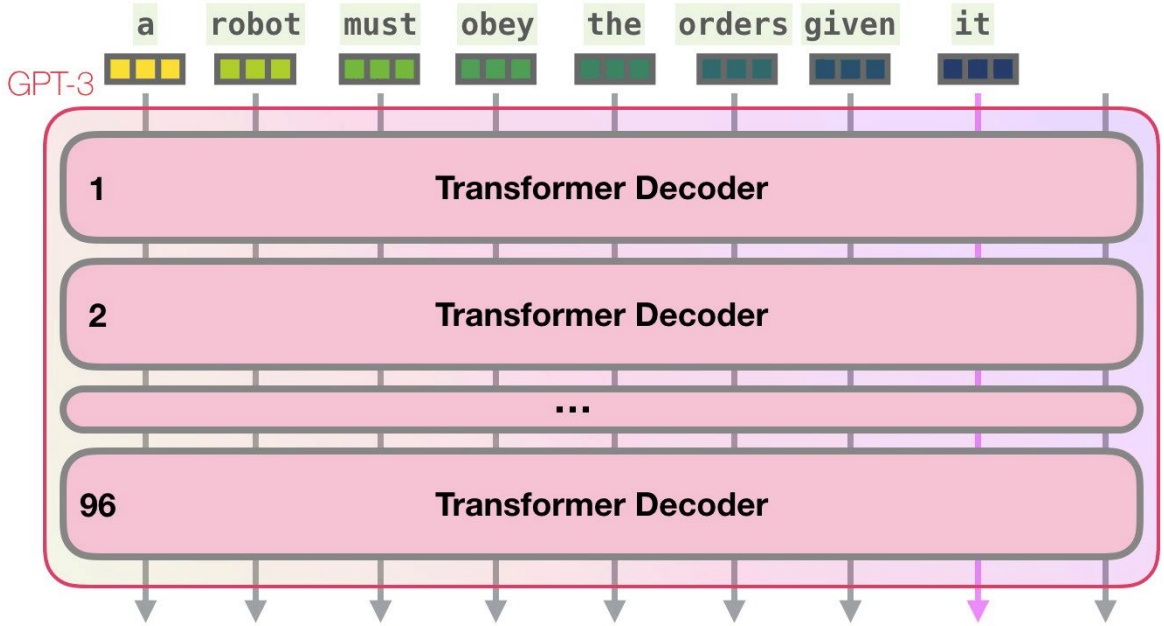
Does the modality matter?

Let's recall how text generation worked:



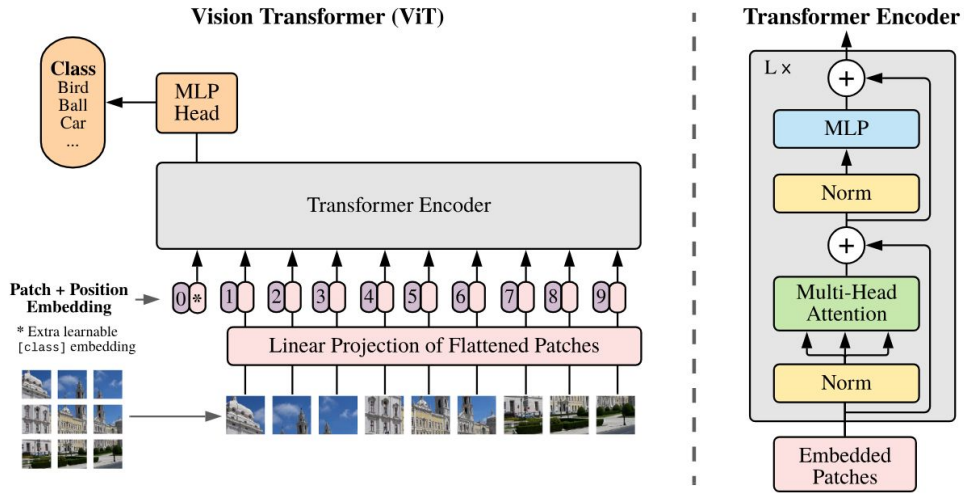
Does the modality matter?

Let's recall how text generation worked (Could be just decoder, like GPT):



Does the modality matter?

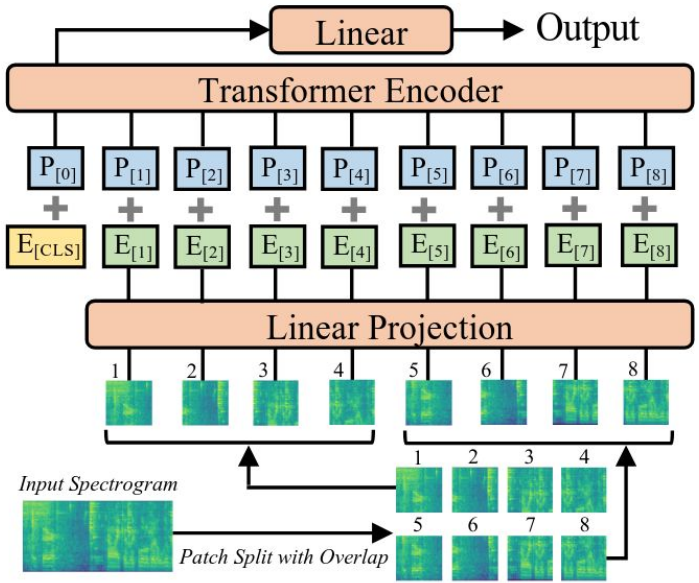
For vision, we would have Vision Transformer (ViT) instead (classification example):



An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale (ICLR 21)

Does the modality matter?

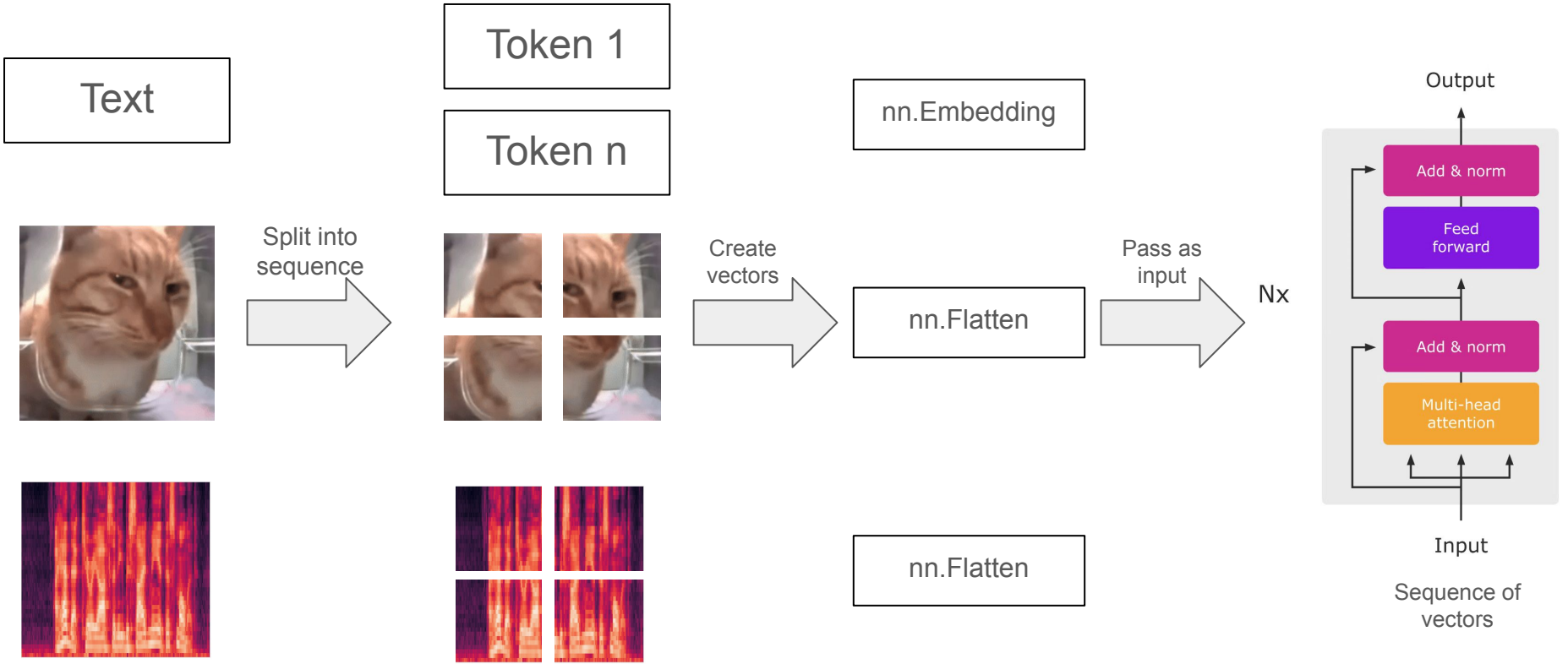
For audio, we would have Audio Spectrogram Transformer (AST) instead (classification example):



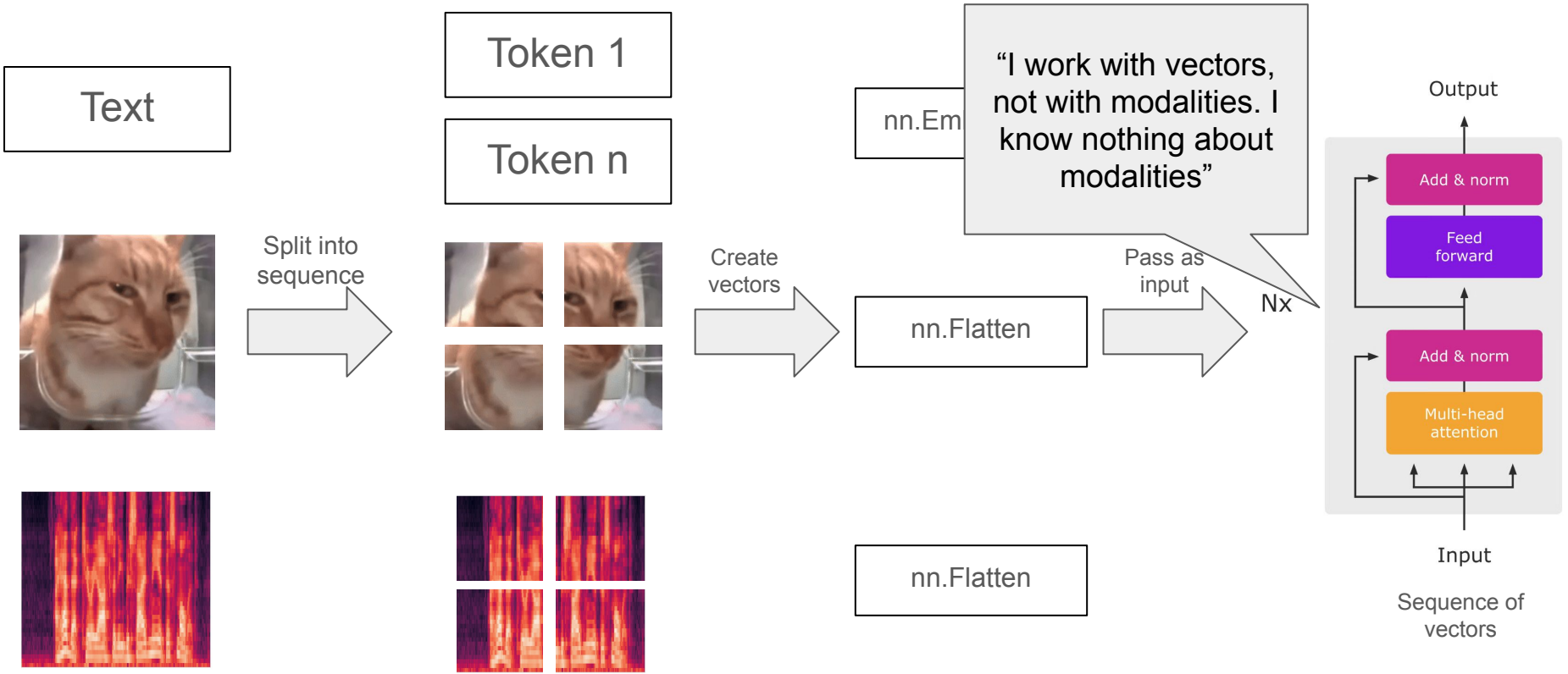
Note: Could take each spectrogram frame as a token embedding too

AST: Audio Spectrogram Transformer (Interspeech 21)

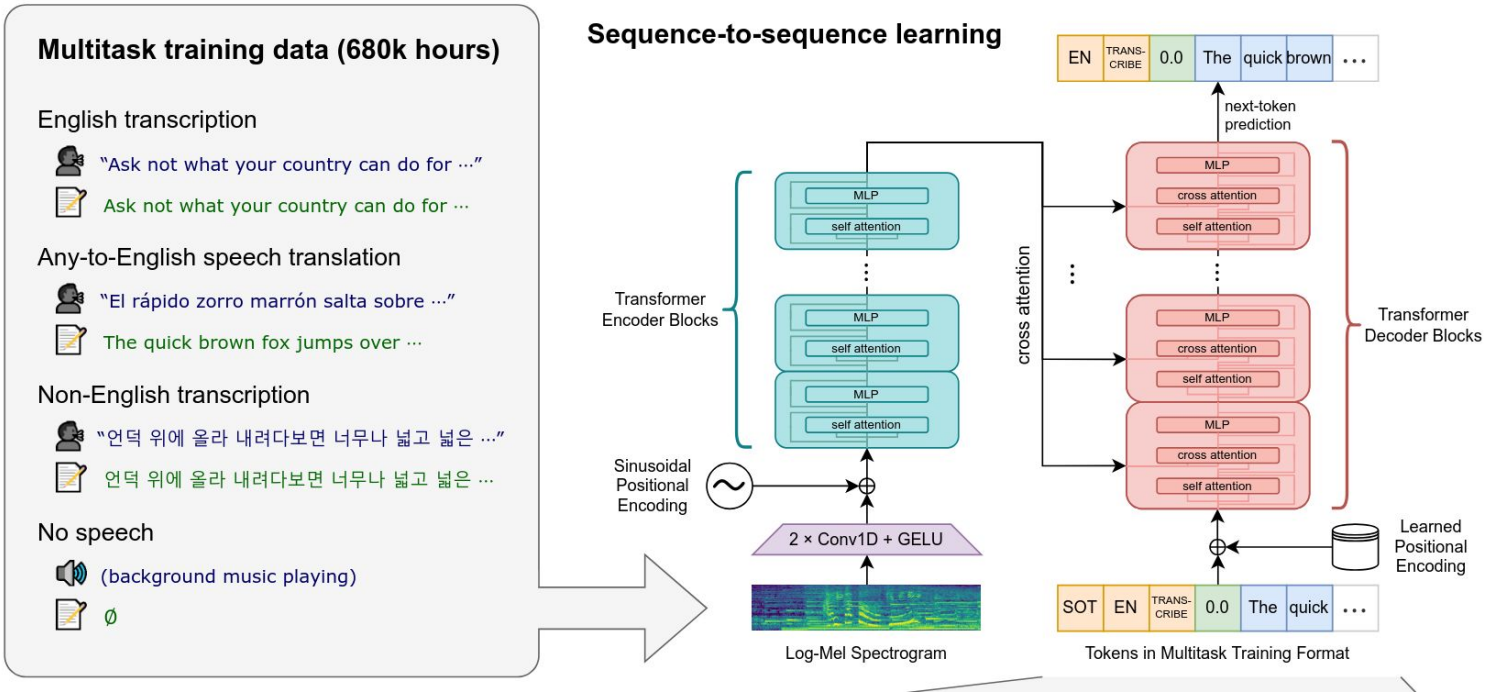
Conclusion: modality is just an abstraction



Conclusion: modality is just an abstraction

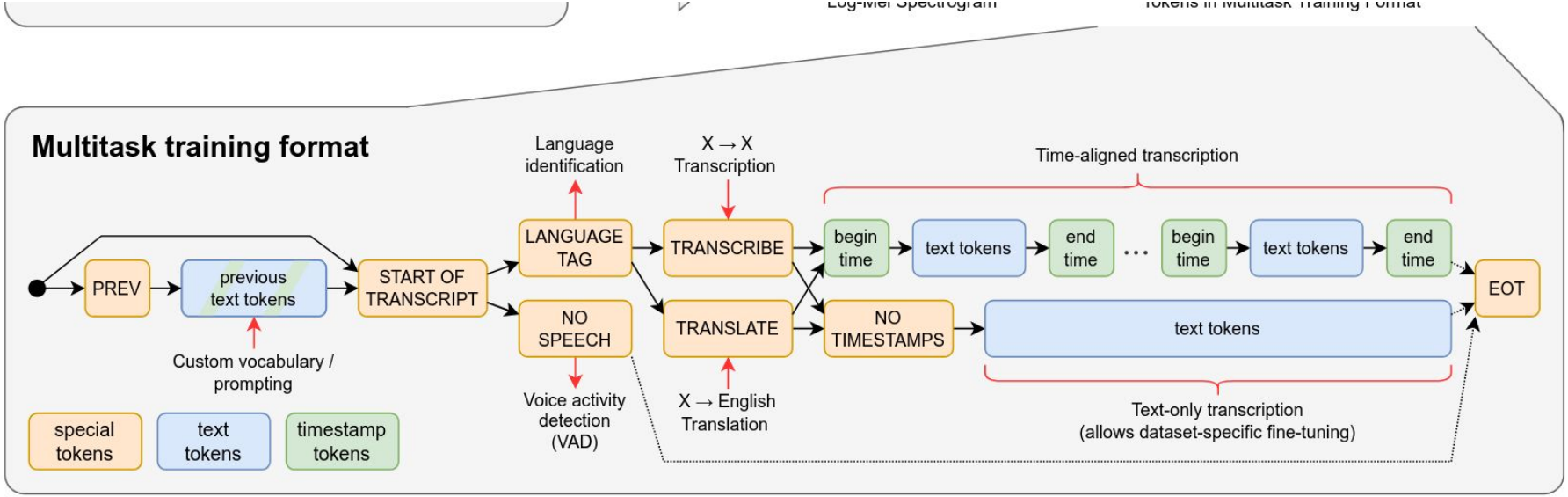


Cross-Entropy: Autoregressive Style of ASR; Whisper



[Robust Speech Recognition via Large-Scale Weak Supervision (ICML 23)]

Cross-Entropy: Autoregressive Style of ASR; Whisper



Robust Speech Recognition via Large-Scale Weak Supervision (ICML 23)